Programación lineal

Geometría, método del símplex y precios sombra

Oswaldo José Velásquez Castañón

Métodos Numéricos – 2026-2

El plan de producción de BIM

BIM fabrica dos tipos de chips. Cada chip de tipo 1 deja un margen de 12 y cada chip de tipo 2 deja un margen de 9.

Los insumos son limitados. Un chip de tipo 1 usa el doble de cobre que uno de tipo 2.

¿Qué mezcla maximiza el margen total?

Insumo	Tipo 1	Tipo 2	Límite
Silicio	1	0	1000
Germanio	0	1	1500
Plástico	1	1	1750
Cobre	4	2	4800

Los coeficientes indican el consumo por chip en las unidades de cada insumo.

Postek et al. (2025), cuaderno 2.1

Variables, objetivo y restricciones

x_1, x_2 son las cantidades de chips de cada tipo. El modelo continuo es

$$
\begin{aligned}
\text{máx} \quad & 12x_1 + 9x_2 & \\
\text{sujeto a} \quad & x_1 \le 1000 & \text{(silicio)},\\
& x_2 \le 1500 & \text{(germanio)},\\
& x_1 + x_2 \le 1750 & \text{(plástico)},\\
& 4x_1 + 2x_2 \le 4800 & \text{(cobre)},\\
& x_1, x_2 \ge 0. &
\end{aligned}
$$

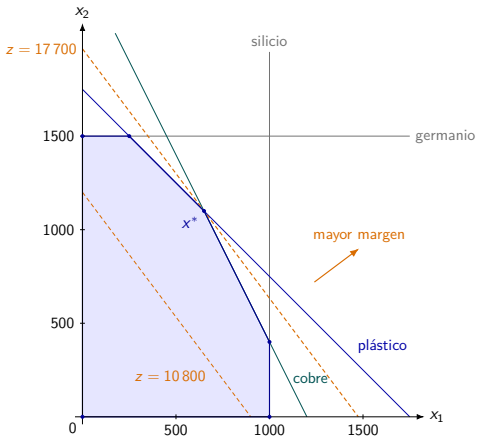
En forma matricial, $\max\{c_0^\top x : A_0 x \le b,\ x \ge 0\}$. La holgura $b_i - (A_0 x)_i$ mide cuánto queda del insumo i.

Región factible y rectas de igual margen

Cada restricción conserva un semiplano. Su intersección es el polígono sombreado.

La recta $12x_1 + 9x_2 = z$ se desplaza hacia valores mayores de z. El último contacto ocurre donde se agotan el plástico y el cobre.

$$x^* = (650, 1100),$$
$$z^* = 17\,700.$$



Vértices óptimos y empates

En un polígono factible acotado, cada plan es un promedio ponderado de los vértices. El objetivo lineal toma el mismo promedio de sus valores.

$$x = \sum_i \theta_i v_i, \qquad c_0^\top x = \sum_i \theta_i \, c_0^\top v_i, \qquad \theta_i \geq 0, \quad \sum_i \theta_i = 1.$$

Al menos un vértice es óptimo.

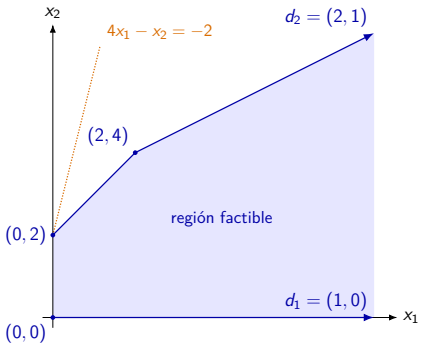
Si el objetivo de BIM fuera máx$(x_1 + x_2)$, cualquier punto de la arista entre $(250, 1500)$ y $(650, 1100)$ alcanzaría 1750. En ambos extremos y entre ellos se agota el plástico.

Regiones no acotadas

La región satisface $x_1, x_2 \geq 0$, $-x_1 + x_2 \leq 2$ y $-x_1 + 2x_2 \leq 6$. Sus dos rayos permiten avanzar indefinidamente.

Minimizar $x_1 - 3x_2$**.** Cada paso en $(2, 1)$ reduce el costo en 1.

Minimizar $4x_1 - x_2$**.** Ambos rayos aumentan el costo. El mínimo vale -2 en $(0, 2)$.



Adaptado del ejemplo 3.1 del material *Método del Simplex: Fundamentos y Pseudocódigo*.

Forma estándar y base inicial

En la notación de Bazaraa, la forma estándar es

$$\min_{x \geq 0} \; c^{\top} x \qquad \text{sujeto a} \qquad Ax = b, \qquad A \in \mathbb{R}^{m \times n}, \quad \operatorname{rang}(A) = m.$$

Para BIM, x_1, x_2 son los niveles de producción y $x_3, \ldots, x_6$ son las holguras de silicio, germanio, plástico y cobre, en ese orden.

$$A = [A_0 \mid I], \qquad c = \begin{pmatrix} -c_0 \\ 0 \end{pmatrix}.$$

Plan inicial

Con $x_1 = x_2 = 0$, las holguras valen b. La matriz básica inicial es $B = [a_3 \mid a_4 \mid a_5 \mid a_6] = I$, donde a_j es la columna j de A.

Soluciones básicas factibles

Reordenamos las columnas de A para reunir en B las m columnas elegidas, con B invertible. Las demás forman N.

$$A = [B \mid N], \qquad Bx_B + Nx_N = b, \qquad x_N = 0 \implies x_B = B^{-1}b.$$

Las variables de x_B son **básicas**. Las de x_N son **no básicas**. La solución básica es factible si $x_B \geq 0$. Las soluciones básicas factibles son los vértices. Si alguna básica vale cero, el vértice es degenerado.

Ejemplo 3.2

Con $x_1 + x_2 + x_3 = 6$ y $x_2 + x_4 = 3$, elegir $B = [a_1 \mid a_2]$ fija $x_3 = x_4 = 0$. Se obtiene $x = (3, 3, 0, 0)$, que se proyecta en el vértice $(3, 3)$ del plano (x_1, x_2).

Costos reducidos y variable entrante

Desde la solución básica factible $x_B = B^{-1}b$, $x_N = 0$, despejamos las básicas al permitir que cambie x_N.

$$Bx_B + Nx_N = b \quad \Longrightarrow \quad x_B = B^{-1}b - B^{-1}Nx_N.$$

Al sustituir en el objetivo, con R el conjunto de índices no básicos,

$$z = c_B^\top x_B + c_N^\top x_N = z_0 + \sum_{j \in R} \bar{c}_j x_j,$$

$$z_0 = c_B^\top B^{-1} b, \qquad z_j = c_B^\top B^{-1} a_j, \qquad \boxed{\bar{c}_j = c_j - z_j}.$$

El costo reducido $\bar{c}_j$ mide el cambio por unidad de x_j. Si $\bar{c}_j < 0$, conviene aumentar x_j siempre que el paso sea factible. Si $\bar{c}_j \geq 0$ para todo $j \in R$, la solución básica actual es óptima.

Cociente mínimo y primer pivote de BIM

Al aumentar solo la entrante x_k, la básica de la fila i queda en

$$x_{B_i} = \bar{b}_i - y_{ik} x_k, \qquad \bar{b} = B^{-1} b, \qquad y_{ik} = (B^{-1} a_k)_i.$$

Solo disminuyen las básicas con $y_{ik} > 0$. Para conservar $x_B \geq 0$,

$$0 \leq x_k \leq \bar{x}_k = \min_{i : y_{ik} > 0} \frac{\bar{b}_i}{y_{ik}}.$$

En BIM, $\bar{c}_1 = -12$ y $\bar{c}_2 = -9$. Bland elige $k = 1$. Como $B = I$, $B^{-1} a_1 = (1, 0, 1, 4)^\top$ y

$$\bar{x}_1 = \min\{1000/1,\ 1750/1,\ 4800/4\} = 1000.$$

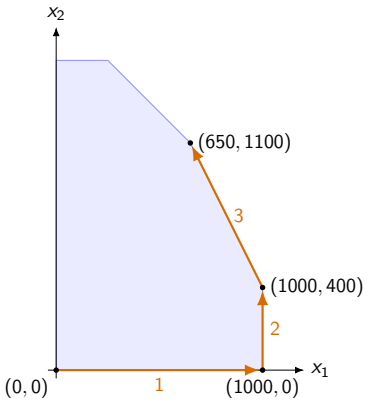
El mínimo está en la fila $r = 1$. Sale x_3 y a_1 sustituye a a_3 en B.

Los tres pivotes de BIM

Paso	Plan (x_1, x_2)	Margen
0	$(0, 0)$	0
1	$(1000, 0)$	12 000
2	$(1000, 400)$	15 600
3	$(650, 1100)$	17 700

En el primer pivote, la nueva base es $[a_1 \mid a_4 \mid a_5 \mid a_6]$.

En el tercer pivote, reducir x_1 en uno libera cobre para aumentar x_2 en dos. El margen sube en $2(9) - 12 = 6$ por sustitución, hasta agotar el plástico.



Algoritmo del símplex revisado

1. Resolver $B\bar{b} = b$ y calcular $\bar{c}_j = c_j - z_j$ para $j \in R$.
2. Si todos los $\bar{c}_j \geq 0$, devolver la solución básica actual.
3. Elegir k con $\bar{c}_k < 0$. Obtener $y_{ik} = (B^{-1}a_k)_i$. Si todos los $y_{ik} \leq 0$, el costo puede disminuir sin límite.
4. Calcular $\bar{x}_k$ y la fila saliente r. Sustituir a_{B_r} por a_k en B y repetir. Si $\bar{x}_k = 0$, el punto no se mueve.

Regla de Bland. Elegir el menor índice elegible para entrar y el menor índice básico si hay empate al salir.

Fase I. Si falta una base factible, se minimiza la suma de variables artificiales. Un mínimo positivo certifica infactibilidad.

El código evalúa las expresiones con B^{-1} mediante sistemas lineales.

Certificado dual y precios sombra

Volvemos al máximo original de BIM. Si $\lambda \geq 0$ y $A_0^\top \lambda \geq c_0$, cada plan factible satisface

$$c_0^\top x \leq \lambda^\top A_0 x \leq b^\top \lambda.$$

El dual minimiza $b^\top \lambda$ sobre estos valores unitarios.

Para BIM, $\lambda = (0, 0, 6, 1.5)$ da la cota $6(1750) + 1.5(4800) = 17\,700$. El plan $(650, 1100)$ alcanza esa cota.

Una unidad adicional de plástico

Con $b_3 = 1751$, el nuevo plan es $(649, 1102)$ y el margen sube a $17\,706$. El precio sombra de 6 mide ese aumento local.

Postek et al. (2025), cuaderno 2.6

BIM con `linprog` y HiGHS

```python
from scipy.optimize import linprog

A = [[1, 0], [0, 1], [1, 1], [4, 2]]
b = [1000, 1500, 1750, 4800]
res = linprog([-12, -9], A_ub=A, b_ub=b,
              bounds=(0, None), method="highs")
if not res.success:
    raise RuntimeError(res.message)

print(res.x, -res.fun)          # [650. 1100.] 17700.0
print(-res.ineqlin.marginals)   # [0. 0. 6. 1.5]
```

SciPy minimiza. Se cambia el signo del objetivo al ingresar los datos y el de los marginales al interpretar el máximo.

Documentación de linprog

Demostración en Python

Abre el cuaderno en Colab y selecciona **Entorno de ejecución** → **Ejecutar todas**.

En la sección BIM del cuaderno:

1. Sigue los pivotes del símplex en la región factible.
2. Compara el plan final y el margen con HiGHS.
3. Revisa el gráfico del margen al variar la disponibilidad de plástico.

Para discutir. Si se añade una unidad de silicio al modelo original, ¿cambia el margen máximo? ¿Cómo se puede comprobar con las holguras y el certificado dual?

Instalación de Pyomo y HiGHS

La primera celda del cuaderno de Colab instala las bibliotecas. Pyomo expresa el modelo y HiGHS calcula la solución.

```
%pip install -q numpy scipy matplotlib pyomo highspy
```

La sección LAD verifica la disponibilidad de HiGHS desde Pyomo.

```
import pyomo.environ as pyo


def crear_solver():
    solver = pyo.SolverFactory("appsi_highs")
    if not solver.available(exception_flag=False):
        raise RuntimeError("HiGHS requiere el paquete highspy.")
    return solver
```

Instalación y solvers del libro

Resolución y lectura de resultados

```
def resolver(modelo, solver):
    resultado = solver.solve(modelo, load_solutions=False)
    pyo.assert_optimal_termination(resultado)
    modelo.solutions.load_from(resultado)
```

- `load_solutions=False` permite revisar cómo terminó el solver antes de cargar valores, incluso si el modelo es infactible.
- `assert_optimal_termination` detiene la ejecución si el solver no reporta un óptimo para estos problemas lineales.
- Tras cargar la solución, usar `pyo.value(...)` para consultar variables, expresiones u objetivos.

Antes de resolver, `modelo.pprint()` muestra los componentes del modelo.

Regresión LAD

LAD (*Least Absolute Deviation*) es la regresión de mínima suma de desviaciones absolutas. Para el hogar i, y_i es el gasto y z_i es el vector de regresores.

$$\begin{aligned} \text{mín} \quad & \sum_i (p_i + q_i) \\ \text{sujeto a} \quad & \alpha + z_i^\top \beta + p_i - q_i = y_i \quad (\text{cada } i), \\ & p_i, q_i \geq 0, \qquad \alpha, \beta_j \text{ libres.} \end{aligned}$$

En el óptimo, p_i y q_i son las partes positiva y negativa del residuo. Si ambas fueran positivas, se podrían reducir por igual manteniendo el ajuste. Con solo un intercepto, el ajuste LAD estima una mediana.

Postek et al. (2025), cuaderno 2.2

LAD en Pyomo. Datos y variables

Cinco observaciones sintéticas de ingreso z y gasto y. La observación central tiene un gasto atípico de 15.

```
z = (0, 1, 2, 3, 4)
y = (1, 3, 15, 7, 9)
m = pyo.ConcreteModel("LAD")
m.I = pyo.RangeSet(0, len(y) - 1)
m.alpha = pyo.Var(domain=pyo.Reals)
m.beta = pyo.Var(domain=pyo.Reals)
m.p = pyo.Var(m.I, domain=pyo.NonNegativeReals)
m.q = pyo.Var(m.I, domain=pyo.NonNegativeReals)
```

`RangeSet` incluye ambos extremos. `Reals` deja libres el intercepto y la pendiente. Las desviaciones son no negativas.

LAD en Pyomo. Ajuste y resultado

```
@m.Constraint(m.I)
def residuo(m, i):
    return m.alpha + z[i] * m.beta + m.p[i] - m.q[i] == y[i]

m.error = pyo.Objective(
    expr=sum(m.p[i] + m.q[i] for i in m.I), sense=pyo.minimize
)


lad = modelo_lad()
resolver(lad, solver)
print(f"Intercepto = {pyo.value(lad.alpha):.2f}")
print(f"Pendiente  = {pyo.value(lad.beta):.2f}")
print(f"Error LAD  = {pyo.value(lad.error):.2f}")
```

Resultado. $\hat{y} = 1 + 2z$. Solo el dato con $z = 2$ deja un residuo de $15 - 5 = 10$.

Cartera MAD

MAD (*Mean Absolute Deviation*) significa desviación absoluta media.

w_j es el peso del activo j, r_{tj} su retorno observado en t y $\bar{r}_j$ su retorno medio muestral.

$$\begin{aligned} \text{mín} \quad & \frac{1}{T}\sum_{t=1}^{T}(p_t + q_t) \\ \text{sujeto a} \quad & p_t - q_t = \sum_j (r_{tj} - \bar{r}_j) w_j \quad (\text{cada } t), \\ & \sum_j \bar{r}_j w_j \geq R, \qquad \sum_j w_j = 1, \\ & 0 \leq w_j \leq u_j, \qquad p_t, q_t \geq 0. \end{aligned}$$

Se minimiza el MAD de los retornos de la cartera respecto de su media. R fija el retorno medio mínimo y u_j limita el peso del activo j.

Postek et al. (2025), cuaderno 2.3

MAD en Pyomo. Datos, pesos y meta

Retornos sintéticos de dos activos. En Python, `0.03` representa 3 %.

```
r = {"A": (0.00, 0.02, 0.06, 0.08),
     "B": (0.04, 0.03, 0.01, 0.00)}
media = {j: sum(serie) / len(serie) for j, serie in r.items()}

m = pyo.ConcreteModel("Cartera MAD")
m.J = pyo.Set(initialize=list(r))
m.T = pyo.RangeSet(0, len(r["A"]) - 1)
m.R = pyo.Param(initialize=0.03, mutable=True)
m.w = pyo.Var(m.J, bounds=(0, 0.8))
m.p = pyo.Var(m.T, domain=pyo.NonNegativeReals)
m.q = pyo.Var(m.T, domain=pyo.NonNegativeReals)
```

`mutable=True` permite cambiar R. Con el tope de peso, el retorno medio máximo es $0.8(4\,\%) + 0.2(2\,\%) = 3.6\,\%$. Una meta de 4 % es infactible.

MAD en Pyomo. Restricciones y objetivo

```
@m.Constraint(m.T)
def desvio(m, t):
    return m.p[t] - m.q[t] == sum(
        (r[j][t] - media[j]) * m.w[j] for j in m.J
    )

m.presupuesto = pyo.Constraint(expr=sum(m.w.values()) == 1)
m.retorno = pyo.Expression(expr=sum(media[j] * m.w[j] for j in m.J))
m.meta = pyo.Constraint(expr=m.retorno >= m.R)
m.riesgo = pyo.Objective(
    expr=sum(m.p[t] + m.q[t] for t in m.T) / len(m.T),
    sense=pyo.minimize,
)
```

El decorador genera una igualdad por período.

MAD en Pyomo. Cambio de la meta

```
mad = modelo_mad()
for meta in (0.03, 0.035):
    mad.R.set_value(meta)
    resolver(mad, solver)
    pesos = {j: round(pyo.value(mad.w[j]), 6) for j in mad.J}
    print(f"\nMeta = {meta:.1%}")
    print("Pesos:", pesos)
    print(f"Retorno medio = {pyo.value(mad.retorno):.3%}")
    print(f"Riesgo MAD    = {pyo.value(mad.riesgo):.3%}")
```

Meta R	Peso A	Peso B	MAD (puntos porcentuales)
3.0 %	50 %	50 %	0.750
3.5 %	75 %	25 %	1.875

Al subir el peso de A, se compensan menos las variaciones de ambos activos.

Material de consulta

- Notas de Métodos Numéricos, capítulo 9, sección de programación lineal.
- *Método del Simplex: Fundamentos y Pseudocódigo*, material del curso Optimización para Machine Learning basado en Bazaraa, Jarvis y Sherali.
- Postek, Zocca, Gromicho y Kantor (2025). *Hands-On Mathematical Optimization with Python*. Cambridge University Press.

Producción BIM Dual de BIM Regresión LAD Cartera MAD

Cuaderno de demostración en Colab